포트폴리오

이수현

e-mail: isuhyeon923@gmail.com

git-hub: https://github.com/gradeF

목차

프로젝트 목록 2

NVIDIA Omniverse POC 4

MROSystems 10

메타버스 기반 산업단지 고도화 사업 17

MetaMaker 22

DEUS EX MACHINA 25

Windows API를 이용한 쿠키런 모작 41

NIGHTMARE 50

프로젝트 목록

NVIDIA Omnivere POC

프로젝트 기간: 2025년 하반기

프로젝트 인원: 2인(프로그래머 2인)

실시간 협업 환경 및 웹 기반 고성능 스트리밍 시스템 구축

사용 엔진: Python, OpenUSD, NVIDIA Omniverse, git

MRO Systems

프로젝트 기간: 2025 상반기

프로젝트 인원: 4인(프로그래머 4인, 기획 3인)

VR 기반 자산 관리 시스템 개발

사용 엔진: Unity 3D, C#

메타버스 기반 산업단지 고도화 사업 -

https://youtu.be/aVFisJWerbA?si=qTLRDT2Jf_Cs5TrO

프로젝트 기간: 2023 - 2025 (3년)

프로젝트 인원: 2인(프로그래머 2인, 기획 3인)

메타버스 기반 협업 시스템

사용 엔진: Unity 3D, C#, Photon, WebRTC, MRTK API, SVN, git

MetaMaker - https://youtu.be/gUf8SdJBZFc

프로젝트 기간: 2023년 하반기

프로젝트 인원: 4인(프로그래머 4인, 기획 3인)

메타버스 콘텐츠 저작 도구 개발

사용 엔진: Unity, C#, SVN, CSV

DEUS EX MACHINA - https://youtu.be/gUf8SdJBZFc

프로젝트 기간: 2022년 하반기

프로젝트 인원: 8인(프로그래머 3인, 기획 5인)

Dead by Daylight 모작

사용 엔진: Unity 3D, C#

Windows API를 이용한 쿠키런 모작 - https://youtu.be/3R8aKTtWkls

프로젝트 기간: 2022년 중반기

프로젝트 인원: 1인

쿠키런 오븐브레이크 모작

사용 언어: C++

NIGHTMARE - https://youtu.be/dx0Je1bFNW0

https://www.dbpia.co.kr/Journal/articleDetail?nodeId=NODE07274831

프로젝트 기간: 2017년 상반기

프로젝트 인원: 3인

VR 탈출 게임

사용 엔진: Unity3D, C#

NVIDIA Omniverse POC

프로젝트 기간: 2025년 하반기

프로젝트 인원: 3인
(프로그래머 2인, 기획 1인)

실시간 협업 환경 및 웹 기반 고성능
스트리밍 시스템 구축

사용 엔진: NVIDIA Omniverse,
python, OpenUSD

수행 기간

2025.10 ~ 2025.11

역할 및 직책

클라이언트 프로그래머/ Omniverse 아키텍처 및 스트리밍 시스템 구축

사용 기술

Python, OpenUSD, NVIDIA Omniverse, git

프로젝트 개요

NVIDIA Omniverse 플랫폼을 활용한 차세대 실시간 협업 환경 검증 및 웹 기반 고성능 스트리밍 시스템 구축을 위한 기술 검증(POC)

핵심 기여 사항

1. Web-Omniverse 실시간 스트리밍 시스템 구축
2. OpenUSD 기반의 데이터 파이프라인 개발
3. Omniverse 확장 프로그램 개발 및 환경 구축

1. Web-Omniverse 실시간 스트리밍 시스템 구축

1.1. 개발 의도

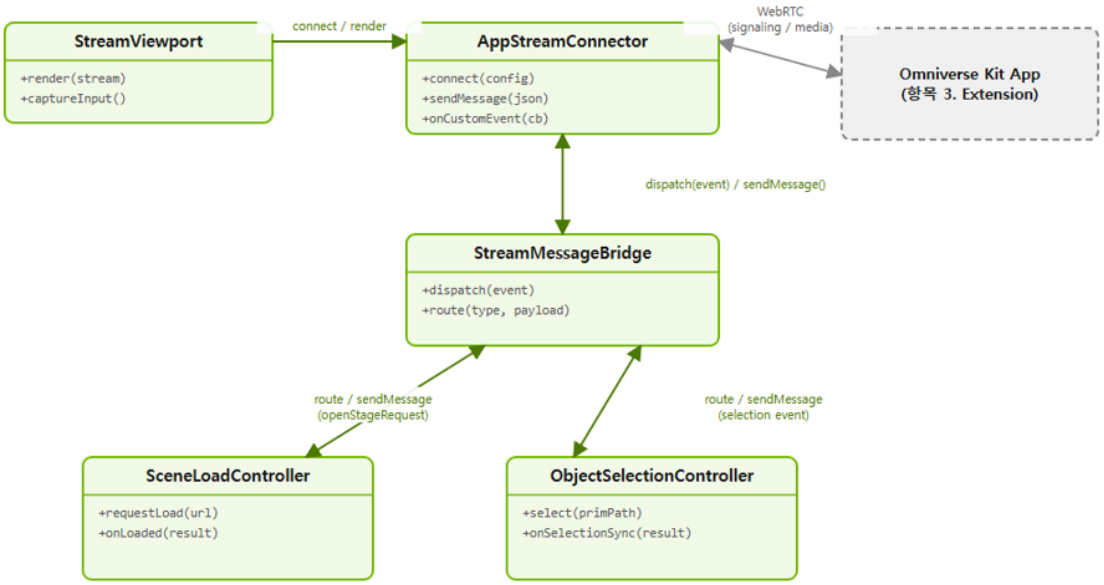
고사양 GPU 없이도 브라우저만으로 Omniverse 3D 씬에 접근하여 실시간으로 확인, 협업할 수 있는 환경이 필요해 개발.
별도의 클라이언트 설치나 3D 뷰어 없이, 웹 브라우저에서 Omniverse Kit 앱을 원격으로 스트리밍 받아 조작함으로써 하드웨어 제약 없는 협업 리뷰 워크플로우를 검증하고자 함.

1.2. 시스템

Nvidia의 web-viewer-sample을 기반으로, 사내 네트워크 환경에서 Omniverse Kit 앱을 WebRTC로 스트리밍하는 구조를 구축함.

- Kit 앱(서버) ↔ WebRTC(omniverse-webrtc-streaming-library, signaling/media 포트 분리) ↔ 자체 제작 웹 클라이언트(React + TypeScript + Vite)
- 클라이언트-서버 간 JSON 기반 커스텀 메시지 프로토콜(event_type + payload)을 확장하여 씬 로드 요청, 오브젝트 선택 동기화 등 양방향 이벤트 처리 구현
- 웹 UI는 샘플의 기본 구조를 참고하되 자체 화면으로 새로 제작

1.3. 클래스 다이어그램



- AppStreamConnector — AppStreamer 래핑, WebRTC 연결/해제 관리
- StreamMessageBridge — 커스텀 메시지 송수신 라우팅 (sendMessage / onCustomEvent 핸들링)
- SceneLoadController — "씬 로드" 요청/응답 처리
- ObjectSelectionController — 오브젝트 선택 이벤트 송수신 및 UI 하이라이트 동기화
- StreamViewport — 스트리밍 영상 렌더링 및 입력(마우스/키보드) 캡처 UI 컴포넌트

1.4. 설명

Local 배포 방식으로 Kit 앱과 웹 클라이언트를 동일 네트워크에서 직접 연결함. 기존 샘플이 제공하는 기본 메시지 체계(openStageRequest 등)를 확장하여, 사용자가 웹 UI에서 특정 오브젝트를 클릭하면 해당 선택 정보가 Kit 앱으로 전달되어 3D 씬 내 오브젝트가 하이라이트되고, 반대로 Kit 앱 측 상태 변경도 웹 클라이언트에 반영되는 양방향 동기화를 구현함. 웹 클라이언트는 샘플 코드를 참고하여 자체 UI/UX로 새로 제작함.

2. OpenUSD 기반의 데이터 파이프라인 개발

2.1. 개발 의도

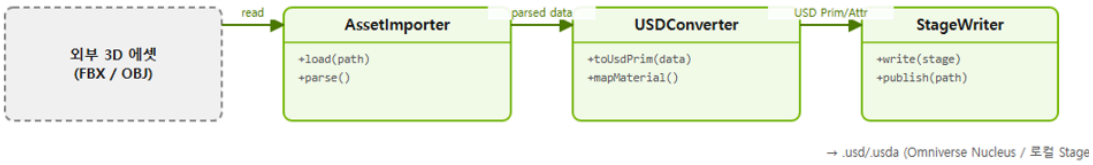
외부에서 제작된 3D 에셋(FBX/OBJ 등 다양한 포맷)을 Omniverse 환경에서 바로 활용하려면 OpenUSD 포맷으로의 변환이 필요함. 수작업 변환/임포트 없이, 다양한 소스 에셋을 일관된 파이프라인을 통해 USD Stage로 편입시켜 협업 및 스트리밍 시스템(1번 항목)에서 곧바로 사용할 수 있도록 하는 것이 목표임.

2.2. 시스템

Python과 OpenUSD API(pxr.Usd, UsdGeom 등)를 활용해 외부 3D 에셋을 USD로 변환하는 파이프라인을 구축함.

- 입력: FBX/OBJ 등 외부 포맷 에셋
- 변환: Python 스크립트가 지오메트리/머티리얼/트랜스폼 등을 파싱하여 USD Prim으로 매핑
- 출력: Omniverse Nucleus/로컬 Stage에 반영되는 .usd/.usda 파일

2.3. 클래스 다이어그램



- AssetImporter — 외부 포맷 파일 로드 및 파싱 진입점
- USDConverter — 파싱된 데이터를 USD Prim/Attribute로 변환
- StageWriter — 변환 결과를 USD Stage 파일로 저장/게시

2.4. 설명

Python 기반 변환 스크립트로 외부 3D 에셋을 읽어 USD Prim 계층 구조로 매핑하고, 지오메트리·머티리얼·트랜스폼 정보를 OpenUSD 스키마에 맞게 변환해 Stage 파일로 출력하는 파이프라인을 구현함. 이를 통해 변환된 에셋은 별도 가공 없이 Omniverse Kit 앱과 웹 스트리밍 뷰어에서 바로 로드하여 확인할 수 있었음.

3. Omniverse 확장 프로그램 개발 및 환경 구축

3.1. 개발 의도

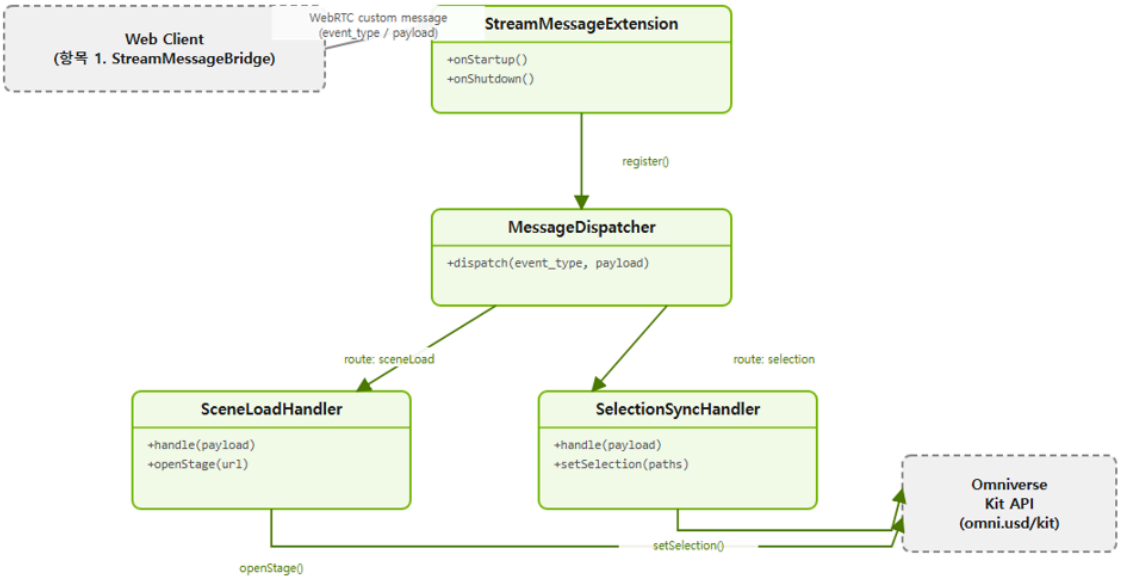
1번 항목의 웹 클라이언트가 보내는 커스텀 스트리밍 메시지(씬 로드 요청, 오브젝트 선택 등)를 Omniverse Kit 앱 내부에서 실제로 처리할 대응 로직이 필요함. 이를 위해 Kit 앱에 탑재되는 자체 Extension을 개발하고, Extension이 정상 동작하는 실행 환경을 구축함.

3.2. 시스템

Kit 앱의 Extension 프레임워크(omni.ext) 위에 커스텀 Extension을 구현하여, 1번 항목의 StreamMessageBridge가 전달하는 JSON 메시지를 수신하고 Omniverse Kit API(Stage 열기, Prim 선택 등)를 호출하는 역할을 담당함.

- 웹 클라이언트 메시지(event_type/payload) 수신
- 메시지 타입별 핸들러 라우팅 후 Kit 앱 내부 동작 실행 (Stage 로드, Selection 변경 등)
- 처리 결과를 다시 웹 클라이언트로 응답(~Result/~Confirmation 메시지) 송신

3.3. 클래스 다이어그램



- StreamMessageExtension — Extension 진입점(IExt 상속), 앱 시작/종료 시 등록/해제
- MessageDispatcher — 수신 메시지 타입별 핸들러로 라우팅
- SceneLoadHandler — 씬 로드 요청 처리 (Stage open)
- SelectionSyncHandler — 오브젝트 선택 요청 처리 및 Kit Selection API 연동

3.4. 설명

Omniverse Kit의 Extension 시스템을 활용해 웹 스트리밍 메시지를 수신하고 처리하는 자체 Extension을 개발함. 웹 클라이언트에서 전달된 씬 로드/오브젝트 선택 요청을 Kit 앱 내부 API 호출로 변환해 실행하고, 처리 결과를 다시 웹 클라이언트에 응답함으로써 1번 항목의 양방향 스트리밍 시스템이 실제로 동작하도록 하는 서버 측 로직을 완성함. 또한 해당 Extension이 Kit 앱 구동 시 자동으로 로드되도록 확장 프로그램 등록 및 개발 환경(Extension 매니페스트, 로컬 개발용 Kit 앱 설정 등)을 구축함.

MROSystems

프로젝트 기간: 2025 상반기

프로젝트 인원: 7인
(프로그래머 4인, 기획 3인)

VR 기반 자산 관리 시스템 개발

사용 엔진: Unity 3D, C#

수행 기간
2025. 01 - 2025.07

역할 및 직책
클라이언트 프로그래머 / UI UX 및 데이터 시스템 고도화 전담

사용 기술
Unity, C#, SVN, CSV

프로젝트 개요
산업 현장의 유지보수(MRO) 효율성을 높이기 위한 VR 기반 자산 관리 시스템 개발 및 데이터 중심의 유연한 콘텐츠 운영 환경 구축

핵심 기여 사항

1. 데이터 중심(Data-Driven) 인벤토리 시스템 설계 및 구현
2. 확장성을 고려한 CSV 데이터 로드 파이프라인 구축
3. 사용자 환경 설정(Preference) 관리 시스템 및 UI 구현

1. 데이터 중심 (Data - Driven) 인벤토리 시스템 설계 및 구현

1.1. 개발 의도

기획자가 csv 파일 수정만으로 프로그램 내 아이템 데이터를 실시간으로 수정할 수 있는 확장성 높고 독립적인 인벤토리 아키텍처 구축함.

1.2. 시스템

데이터 흐름 구조

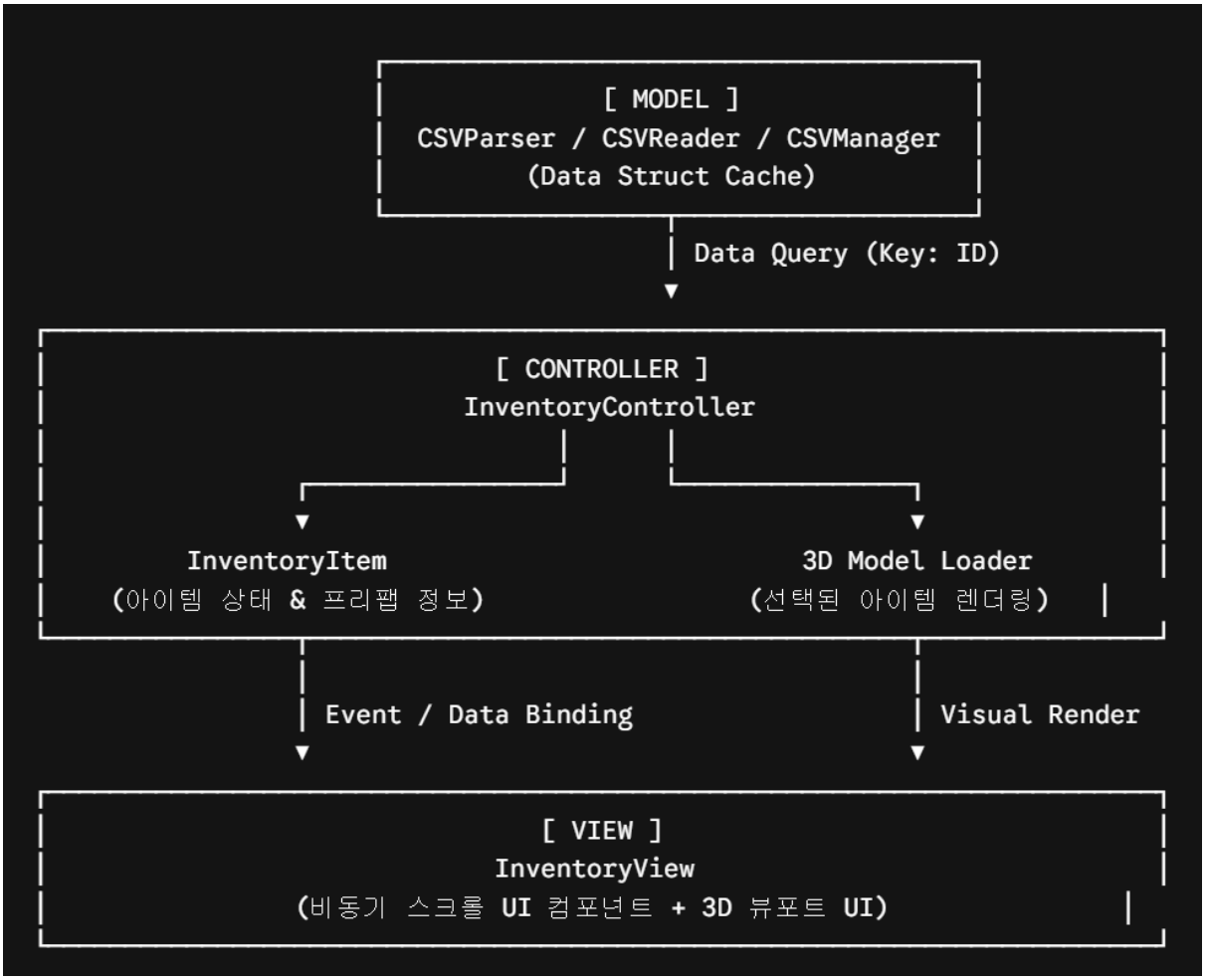
csv 파일 -> csv parser -> data manager -> 인벤토리 시스템 -> 유니티 UI

MVC 패턴을 사용하여 인벤토리 시스템 구축

아이템의 3D 프리팹 시각화

인벤토리 내에서 아이템을 선택했을 때, 해당 아이템에 매핑된 3D 프리팹 모델을 화면에 동적으로 출력하고 시각화하는 기능 구현.

1.3. 클래스 다이어그램



Model

- csvParser: CSV 텍스트 데이터를 읽어 들이는 파싱 클래스
- CSVManager : 파싱된 정적 데이터를 중앙에서 관리하는 클래스
- CSVReader: CSVManager 하위에서 각각의 CSV 파일 구조등에 맞춰 1:1로 대응해 데이터를 읽어오는 클래스
- Data Struct : CSV의 각 항목을 저장하는 테이블 정의 구조체. 이 데이터들을 고유 ID를 Key로 삼어 Dictionary 형태로 받아와 캐싱함.

Controller

- InventoryItem : 인벤토리 슬롯에 들어갈 실제 아이템 객체. CSV에서 읽어들인 데이터를 기반으로 프리팹 정보를 수정하여 인벤토리에 추가.
- InventoryController: 인벤토리의 상태 변화와 데이터 흐름을 총괄하는 코어 로직 클래스.

View

- InventoryView: 타 개발자가 만들어둔 비동기 방식의 스크롤 UI 컴포넌트를 베이스로 활용하여 결합.
- 스크롤 UI는 구조는 수정할 수 없었으나 인벤토리 인터페이스 및 MVC 패턴 기반의 데이터 바인딩 로직은 직접 전담하여 구현
- Controller의 데이터 상태가 변경되어면 UI도 이에 동기화되어 실시간으로 변경되도록 구조화.

1.4. 설명

신규 데이터 클래스 정의만으로 자동 바인딩 되도록 파이프라인 확장성 확보

인벤토리 UI 리뉴얼 작업 시 UI 프리팹과 구조를 완전히 갈아엎더라도 코드 수정 공수가 들지 않는 독립적인 컴포넌트 환경 제공

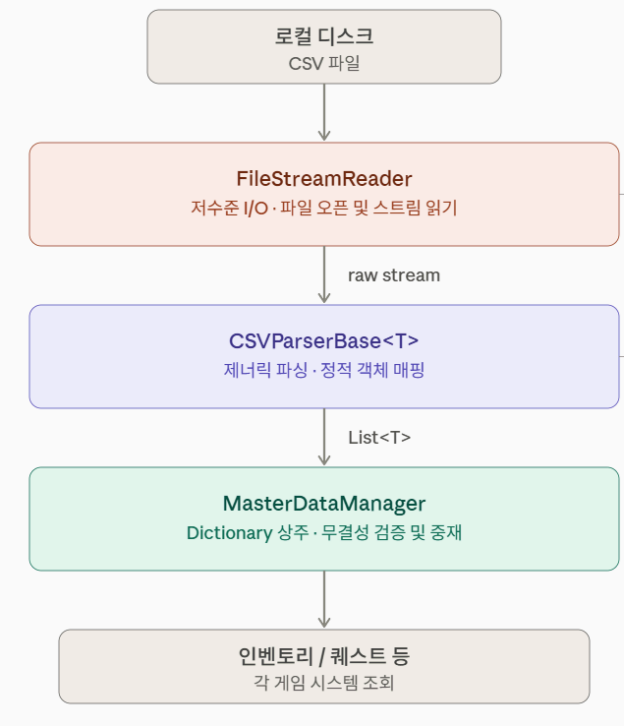
2. 확장성을 고려한 CSV 데이터 로드 파이프라인 구축

2.1. 개발 의도

기획 데이터를 코드와 완전히 격리하여, 개발자의 개입 없이 기획자가 독립적으로 콘텐츠 데이터를 추가하고 제어할 수 있는 공용 파이프라인 형성.

협업 프로젝트로서 개발 효율을 높이기 위해 저수준 파일 처리 레이어와 고수준 파싱/캐싱 레이어의 역할을 나누어 공동 개발 진행.

2.2. 시스템



2.3. 클래스 다이어그램

FileStreamReader : 외부 로컬 디스크에서 CSV 파일을 안전하게 열고 바이트/문자열 스트림으로 읽어오는 저수준 I/O 클래스.

CSVParserBase<T>: 읽어온 스트림을 전달 받아, 제너릭 구조를 기반으로 다양한 형태의 데이터 클래스에 맞춰 정적 C# 객체로 자동 매핑해 주는 공용 파서 클래스.

MasterDataManager: 데이터들을 유일성이 보장된 Dictionary 구조체로 변환하여 메모리에 상주시킨 뒤, 다른 시스템들이 안전하게 접근할 수 있도록 중재하는 중앙 데이터 매니저 클래스.

2.4. 설명

- 역할 분담을 통한 협업 효율 극대화

공동 작업 진행 시 소스 코드가 엉키는 것을 막기 위해 파일 I/O 파트와 데이터 파싱 및 인메모리 관리 파트로 레이어를 철저히 분리함.

- 아키텍처적 안정성 확보

인터페이스 규격을 먼저 정의하고 개발에 착수하여, 내부 파일 읽기 방식이 변경되더라도 본인이 구축한 파서 엔진과 인벤토리 시스템은 아무런 사이드 이펙트(Side Effect) 없이 독립적으로 작동하는 아키텍처적 안정성을 확보함.

- 데이터 무결성 확보

기획자의 오타나 데이터 타입 기입 실수 등으로 인해 런타임 크래시가 발생하는 것을 방지하기 위해 예외 처리 로직을 결합.
오류 발생 시 동료가 구축한 로그 시스템과 연계하여 에러가 발생한 정확한 텍스트 라인(Line)을 추적할 수 있도록 구현함.

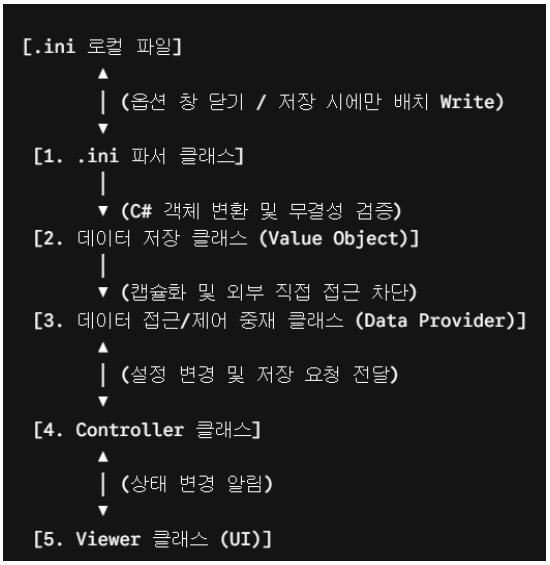
3. 사용자 환경 설정 관리 시스템 및 UI 구현

3.1. 개발 의도

사용자가 변경한 환경 설정 데이터를 유지하고 관리하기 위해서 시스템을 구축함.
섹션 별 데이터 관리가 가능하며 외부에 공개되어 직관적인 편집이 가능한 .ini 파일 포맷을 선택함.
싱글톤 사용을 의도적으로 지양하고 필요한 클래스 내에서만 참조를 유지하는 구조를 취해 결합도를 낮추고 데이터 무결성을 보장함.

3.2. 시스템

5단계 레이어 파이프라인을 구축



3.3. 클래스 다이어그램

1. .ini 파서 클래스
   로컬 디스크의 .ini 파일 텍스트를 읽어 들여 파싱하는 순수 I/O 클래스
2. 데이터 저장 클래스
   파싱된 설정값을 메모리에 보유하는 순수 데이터 컨테이터 객체
3. 데이터 접근/ 제어 중재 클래스
   인터페이스 또는 읽기 전용 프로퍼티를 통해 안전한 조회와 통제된 수정만 허용하는 스토리지 관리자.
4. Controller 클래스
   UI의 조작을 입력 받아 데이터를 반영하고 변경 사항 발생 시 오디오 믹서나 엔진 해상도 시스템에 즉시 동기화를 명령하는 제어 클래스
5. Viewer 클래스
   슬라이더, 토글 등 유저 인터페이스의 화면 표현과 입력을 담당하는 레이어.

3.4. 설명

- 싱글톤 지양을 통한 결합도 최소화

데이터 오염을 방지하기 위해 싱글톤 패턴 배제.

환경 설정 시스템이 필요한 컨텍스트 내부에서만 참조가 전달 되도록 구조화.
시스템 간의 불필요한 결합도 감소 및 모듈의 독립성 확보

- 배치 쓰기를 통한 로컬 디스크 I/O 최적화

런타임에는 인메모리 값만 변경하고 유저가 옵션 창을 닫거나 최종 저장 버튼을 누르는 시점에만 파일 작성을 수행하도록 저장 타이밍을 제어하여 스토리지 부하 최적화

- 이벤트 기반 데이터-UI동기화 (Observer 구조)

Controller 클래스에서 데이터 상태가 변경되면 Viewer가 이를 감지하여 UI 컴포넌트를 자동으로 동기화하도록 이벤트를 바인딩.
UI와 로직이 완전히 분리되어 있어, 향후 UI가 변경되어도 데이터 제어 코어 로직은 영향을 받지 않는 구조로 구성

메타버스 기반 산업단지 고도화 사업



프로젝트 기간: 2023-2025

프로젝트 인원: 3인
(프로그래머 2인, 기획 1인)

메타버스 기반 협업 시스템

사용 엔진: Unity 3D, C#

수행 기간

2023 ~ 2025 (3년)

역할 및 직책

클라이언트 프로그래머

사용 기술

Unity, C#, Photon, WebRTC, MRTK API, SVN, git

프로젝트 개요

사업 현장의 효율적인 협업을 위한 메타버스 기반 실시간 원격 지원 및 AR 가이드 시스템 구축

핵심 기여 사항

1. 시스템 구조 리팩토링을 통한 결합도 해소
2. 실시간 네트워크 및 커뮤니케이션 시스템 구현
3. 절차적 AR 콘텐츠 및 인터렉션 개발

1. 시스템 구조 리팩토링을 통한 결합도 해소

1.1. 개발 의도

여러 서브 시스템을 통합하는 과정에서 프레임드랍이 다수 발생했고, 잦은 싱글톤의 사용으로 기능 수정/보수 시 영향 범위 파악이 어려워 유지보수 문제가 누적됨. 클래스간 결합도를 낮춰 유지보수 가능한 구조로 바꾸기 위함

1.2. 시스템

Manager 싱글톤 -> 이벤트/옵저버 패턴

1.3. 클래스 다이어그램

1.4. 설명

Manager.instance가 씬 전환, 키보드, 번역, 사운드, 네트워크, WebRTC, 드로잉, 이미지/PDF 뷰어를 모두 직접 소유하고 다른 클래스들이 이를 직접 호출하는 구조였음. 이를 이벤트/옵저버 패턴으로 전환하여 매니저 간 직접 참조를 최소화함. 결과적으로 기능 추가에 필요한 소요 시간이 줄어듦.

2. 실시간 네트워크 및 커뮤니케이션 시스템 구현

2.1. 개발 의도

PC - AR 사용자 간의 실시간 스트리밍 화면 공유를 위해 구현함.

2.2. 시스템

Photon(룸 매칭/시그널링, 드로잉 좌표 스트리밍) + WebRTC(영상, 음성, Microsoft MixedReality-WebRTC 기반)

2.3. 클래스 다이어그램

WebRTC 연결 구조

```
classDiagram
    class PeerServer {
        +List~ClientSignaler~ clientSignalerList
    }
    class ClientSignaler {
        +NodeDssSignaler nodeDssSignaler
        +VideoRenderer videoRenderer
```

```
    +Initialized(localPeerId, remotePeerId)
    +ClientCheckConnected()
}
class PeerClient {
    +NodeDssSignaler nodeDssSignaler
    +StartStreamingPeerConnection()
}
class NodeDssSignaler {
    +PeerConnection PeerConnection
    +LocalPeerId
    +RemotePeerId
}
class PeerConnection

PeerServer "1" --> "*" ClientSignaler : 관리
ClientSignaler --> NodeDssSignaler
PeerClient --> NodeDssSignaler
NodeDssSignaler --> PeerConnection
```

2.4. 설명

Photon으로 룸 매칭과 시그널링, 드로잉 좌표를 처리하고, WebRTC로 AR 사용자의 카메라 뷰, 음성을 PC로 실시간 스트리밍함. PeerServer가 접속한 클라이언트별 ClientSignaler를 리스트로 관리하고, 각 시그널러는 NodeDssSignaler를 통해 실제 PeerConnection을 맺음.

WebRTC 네이티브 라이브러리와 빌드 타겟 아티텍처(arm32/arm64) 가 맞지 않아 연결 자체가 맺어지지 않는 문제가 있었고, 빌드 타겟 아키텍처를 고정하여 해결함.

3. 절차적 AR 콘텐츠 및 인터렉션 개발

3.1. 개발 의도

통제된 현장 환경 특성상 관리자가 현장을 직접 왕복하며 대응하는 데 시간이 오래 걸려 즉각적인 문제 해결이 어려움. 현장에 처음 투입되는 작업자가 AR 매뉴얼(절차형 가이드)을 참고해 스스로 순서대로 작업하고, 문제가 생겼을 때만 관리자에게 연락할 수 있도록 하기 위해 개발함.

3.2. 시스템

CSV 기반 데이터 파이프라인(CSV_Manager) + 단계 진행 로직(AR_Manager) + MRTK(핸드 트래킹, 공간 앵커링)

3.3. 클래스 다이어그램

```
classDiagram
    class AR_Manager {
        +int CurStep
        +List~RectTransform~ SpotLights
        +NextStep()
        +PreStep()
        -SetObject()
        -SetScreen()
        -SetExplanation()
        -SetWarning()
    }
    class CSV_Manager {
        +Read(file) List~Dictionary~
        +GetPath()
    }
    AR_Manager --> CSV_Manager : Document
    AR_Manager --> "RawImage ScreenView"
    AR_Manager --> "SpotLights (공간 하이라이트)"
    AR_Manager --> "TMP_Text WarningNotice"
```

3.4. 설명

CSV로 정의된 단계별 데이터(버튼 위치, 화면 이미지, 설명, 경고)CSV_Manager가 읽고, AR_Manager가 CurStep에 따라 순서대로 재생함.

각 단계마다 공간상 특정 위치를 스팟라이트로 하이라이트하고, 화면 이미지를 페이드 전환하며 설명 텍스트를 표시함.

여기에 핸드트래킹 조작과 드로잉 주석 인터랙션이 더해짐. 콘텐츠가 코드에 하드코딩되지 않고 CSV로 분리되어 있어 기획자가 직접 절차 내용을 수정할 수 있음.

MetaMaker

프로젝트 기간: 2023년 하반기

프로젝트 인원: 7인
(프로그래머 4인, 기획 3인)

메타버스 콘텐츠 저작 도구 개발

사용 엔진: Unity 3D, C#

수행 기간

2023.07 ~ 2023.12

역할 및 직책

클라이언트 개발

사용 기술

Unity, C#, git, Json

프로젝트 개요

VR 콘텐츠 제작 공정의 효율화를 위한 메타버스 콘텐츠 저작 도구 개발 및 유연한 데이터 관리 시스템 구축

핵심 기여 사항

1. JSON 기반 데이터 직렬화/역직렬화 시스템 구축
2. VR 콘텐츠 제작 효율화 툴 개발

1. JSON 기반 데이터 직렬화/역직렬화 시스템 구축

1.1. 개발 의도

Unity의 기본 타입(Vector3, Quaternion, Color 등)은 JSON으로 기본 직렬화가 되지 않고, VR 콘텐츠 저작 도구 특성상 오브젝트의 Transform, 머티리얼 색상, 텍스트 폰트/내용 등 다양한 커스텀 속성을 안정적으로 저장하고 복원해야 했음. 기존에 팀원이 구축해 둔 시스템을 이어받아, 저장·불러오기 과정에서의 안정성을 높이는 방향으로 개선을 진행함.

1.2. 시스템

싱글톤으로 구현된 Encoder/Decoder가 여러 데이터 타입을 함께 처리하는 구조입니다. Transform, 머티리얼 색상, 텍스트(폰트/내용) 등 속성별로 직렬화 대상 데이터 클래스를 분리 설계하고, Encoder/Decoder가 이 클래스들을 JSON으로 변환·복원합니다.

1.3. 설명

오브젝트의 속성을 저장할 때 각 속성을 대응하는 데이터 클래스(TransformData, MaterialColorData, TextData 등)로 변환한 뒤 Encoder가 JSON으로 직렬화해 파일로 저장합니다. 불러올 때는 Decoder가 JSON을 파싱해 각 데이터 클래스로 역직렬화한 후 오브젝트에 재적용합니다. 이 구조 덕분에 새로운 속성 타입을 추가하기 쉬워지고, 저장/불러오기 과정의 안정성을 확보할 수 있었음.

2. VR 콘텐츠 제작 효율화 툴 개발

2.1. 개발 의도

코드를 직접 작성하지 않고도 기획자 등 비개발 인력이 VR 콘텐츠의 오브젝트 배치, 씬 구성, 인터랙션 설정까지 할 수 있도록 하여 콘텐츠 제작 공정 전체의 효율을 높이기 위해 개발함.

2.2. 시스템

Unity 에디터 확장과 런타임 UI를 모두 지원하는 구조로, 에디터 상에서 오브젝트를 배치·구성할 수 있고, 클릭/트리거 등 이벤트 기반의 인터랙션도 코드 작성 없이 정의할 수 있음. 오브젝트 배치·구성 기능과 인터랙션 기능을 모두 직접 담당함.

2.3. 설명

사용자는 코드 작성 없이 GUI에서 오브젝트를 배치하고, 어떤 트리거(클릭, 충돌 등)에 어떤 액션(이동, 사운드 재생 등)이 실행될지 인터랙션을 정의합니다. 이 설정은 InteractionData로 저장되며, 런타임에 InteractionRuntime이 이를 읽어 실제 동작으로 실행합니다. 이를 통해 개발 지식이 없는 기획자도 직접 VR 콘텐츠를 제작·수정할 수 있게 되어 제작 공정이 효율화됨.

DEUS EX MACHINA



장르: 비대칭 PvP 서바이벌
(Dead by Daylight 모작)

플레이 인원: 2~5인

엔진: Unity 3D

언어: C#

프로젝트 기간

2022년 하반기

프로젝트 인원

8인(프로그래머 3인, 기획자 5인 구성)

기획의도

비대칭 PvP 게임으로 설정하여 강력한 플레이어에게는 상대방을 압도하는 플레이를, 나약한 다수의 플레이어에게는 시련을 극복하는 성취의 재미를 플레이어에게 선사하도록 기획되었음.

게임 내용

플레이어는 퇴마사와 인형 중 하나의 진영을 선택하여 플레이를 진행함.
퇴마사는 술래잡기의 술래와 같은 역할로 인형 플레이어를 모두 제거하는 것이 목표임.
인형은 퇴마사를 피해 탈출 조건을 만족하고 탈출하는 것이 게임의 목표임.

주 구현 시스템

1. 아이템을 생성하는 랜덤 박스 시스템
2. UI 최적화
3. Object 상호작용 시스템
4. Casting System

1. 확률을 이용해 아이템을 생성하는 랜덤 박스 시스템

1.1. 개발 의도

- 플레이어에게 역전의 발판 혹은 승리를 굳힐 수 있는 기회를 주어 플레이를 다채롭게 하기 위해 아이템 시스템을 기획함.
- 아이템의 등장 확률은 각 아이템에 따른 확률에 의해 무작위로 획득하기 때문에 무작위로 아이템을 생성할 시스템을 개발함.

1.2. 시스템

1.2.1. 시스템 설계

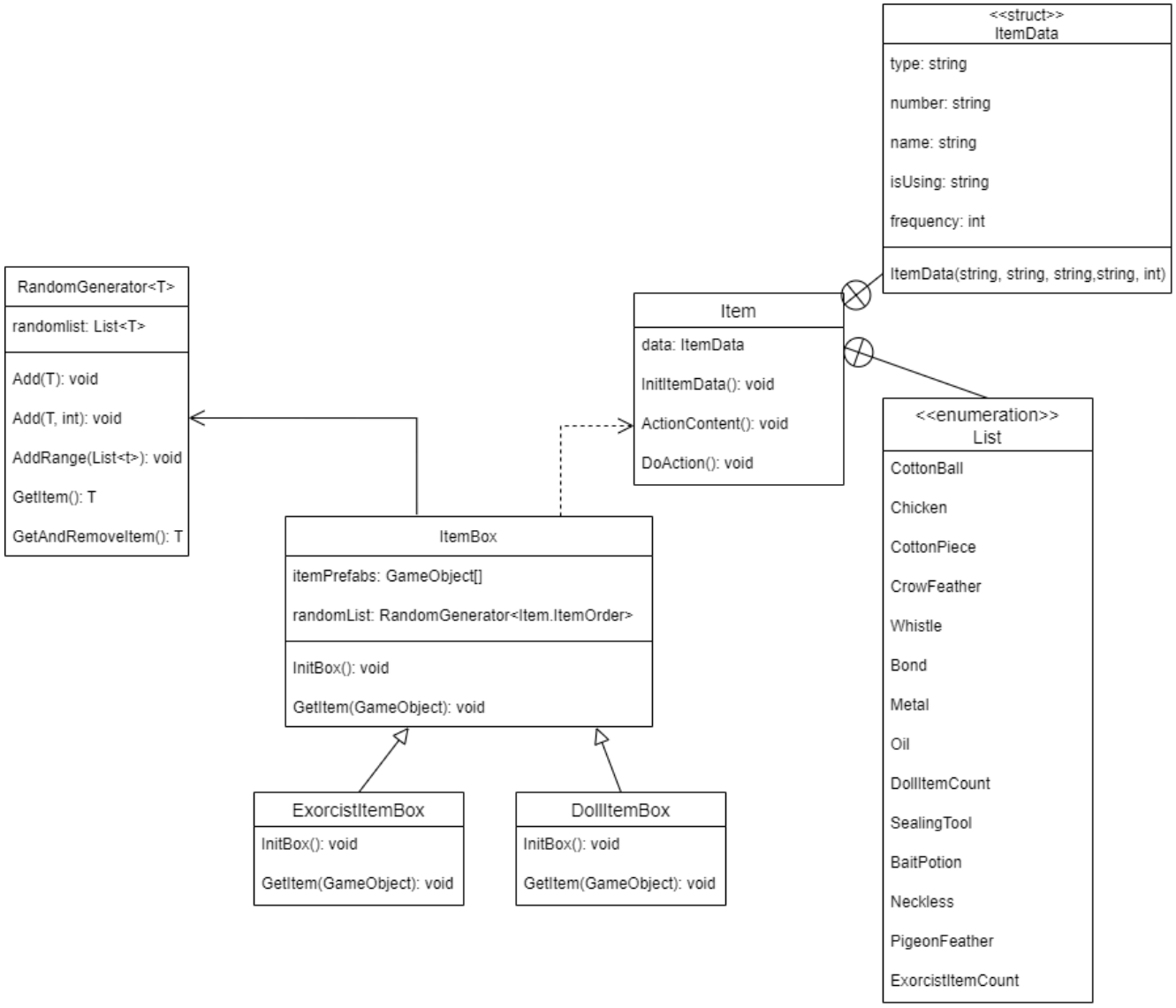
- 모든 아이템은 개별적인 등장 빈도를 가지고 있으며, 아이템 등장 빈도는 아이템의 등장 확률을 계산하는데 사용함.

$$\text{아이템 등장 확률} = \frac{\text{해당 아이템의 등장 빈도}}{\text{모든 아이템의 등장 빈도}}$$

수식 1 아이템 등장 확률 계산법

- 아이템 등장 빈도는 기본 값이 10인 정수 데이터이며, 0 미만의 값을 가질 수 없음.

1.3. 클래스 다이어그램


<<struct>>
ItemData
type: string
number: string
name: string
isUsing: string
frequency: int
ItemData(string, string, string,string, int)
RandomGenerator<T>
randomlist: List<T>
Add(T): void
Add(T, int): void
AddRange(List<t>): void
GetItem(): T
GetAndRemoveItem(): T
Item
data: ItemData
InitItemData(): void
ActionContent(): void
DoAction(): void
<<enumeration>>
List
CottonBall
Chicken
CottonPiece
CrowFeather
Whistle
Bond
Metal
Oil
DollItemCount
SealingTool
BaitPotion
Neckless
PigeonFeather
ExorcistItemCount
ItemBox
itemPrefabs: GameObject[]
randomList: RandomGenerator<Item.ItemOrder>
InitBox(): void
GetItem(GameObject): void
ExorcistItemBox
InitBox(): void
GetItem(GameObject): void
DollItemBox
InitBox(): void
GetItem(GameObject): void

- **RandomGenerator**: 무작위로 오브젝트를 사용할 수 있도록 무작위 리스트를 만들어주는 클래스.
- **Item**: 퇴마사 진형과 인형 진형이 사용할 아이템들을 만들어진 랜덤 리스트에 저장거나 제거하는 클래스.
- **ItemBox**: 아이템 박스를 생성하는 클래스.
- **ExorcistItemBox**: 아이템 박스 클래스를 상속, 퇴마사 진형이 사용하는 아이템 클래스
- **DollItemBox**: 아이템 박스 클래스를 상속, 인형 진형이 사용하는 아이템 클래스

1.4. 설명

```
public class RandomGenerator<T>
    {
        List<T> randomlist = new List<T>();


        public void Add(in T item)
        {
            randomlist.Add(item);
        }
        public void Add(in T item, int count)
        {
            for(var i = 0; i<count; ++i)
            {
                randomlist.Add(item);
            }
        }
        public void AddRange(in List<T> items)
        {
            randomlist.AddRange(items);
        }

        public T GetItem()
        {
            int i = Random.Range(0, randomlist.Count);
```

```
            return randomlist[i];
        }
        public T GetAndRemoveItem()
        {
            int i = Random.Range(0, randomlist.Count);
            T item = randomlist[i];
            randomlist.RemoveAt(i);
            return item;
        }
    }
```

RandomGenerator의 코드

- **Add(in T Item)**: RandomList에 Object를 추가하는 함수.
- **Add(in T Item, int count)**: Count 매개변수를 사용해 원하는 확률만큼 나올 수 있도록 조정함.

  예시) 100개의 아이템에서 25%의 확률로 object를 획득하고 싶다면 25개의 아이템을 리스트에 넣어 25%의 확률로 만듦.
- **GetItem()**: randomlist에서 아이템을 가져오는 함수.
- **GetAndRemoveItem()**: RandomList에서 아이템을 꺼내고 제거하는 함수.

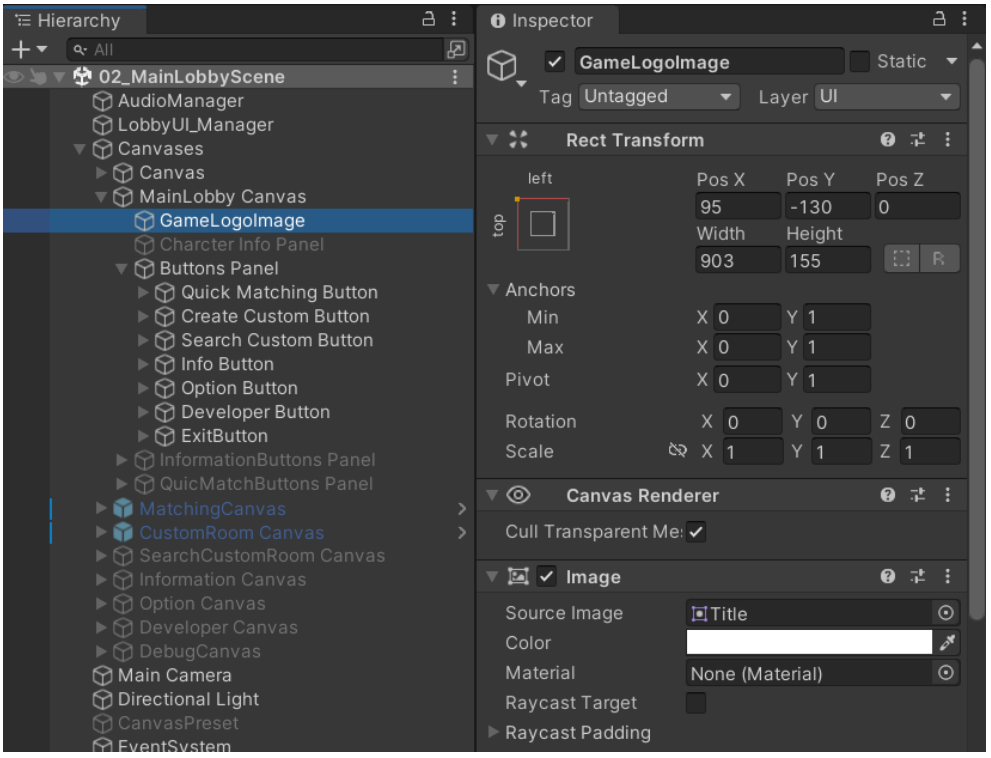
2. UI 최적화

2.1. 개발 의도

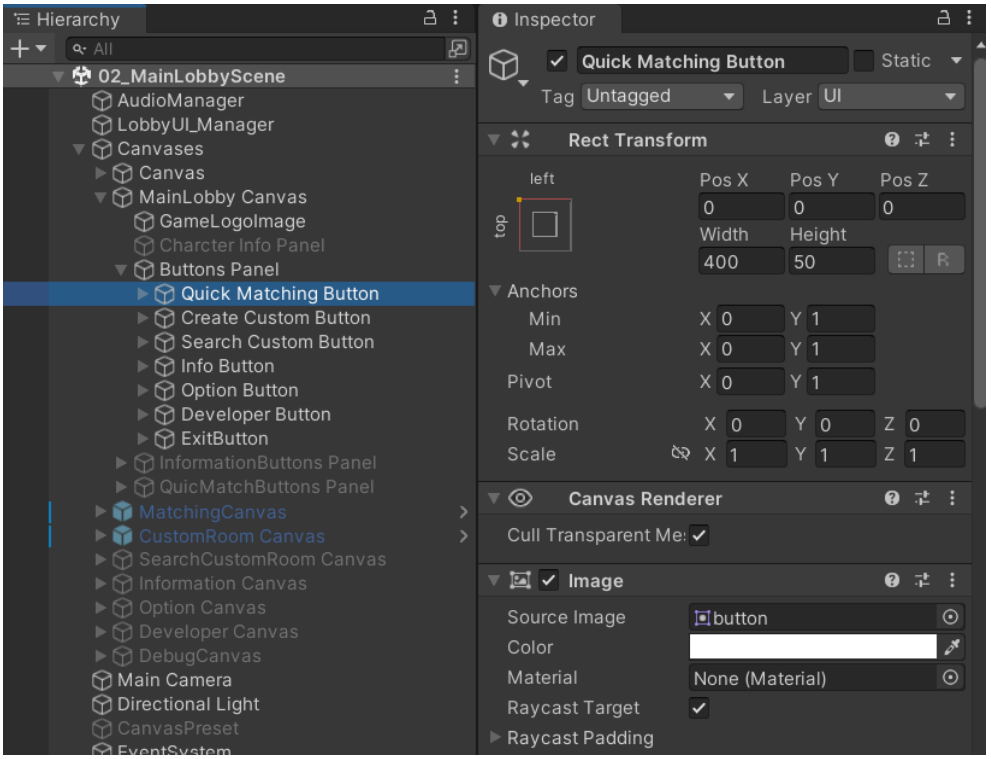
UGUI(Unity UI)는 성능 문제의 원인이 되는 경우가 많으며, Canvas 컴포넌트는 UI요소에 대한 메시를 생성 및 업데이트하고 GPU에 드로우 콜을 보냄. 이러한 기능은 리소스를 많이 소모할 수 있으므로 다음 사항을 적용함.

2.2. Raycast Target 비활성화

상호작용이 없는 UI의 경우 Raycast Target을 비활성화함.



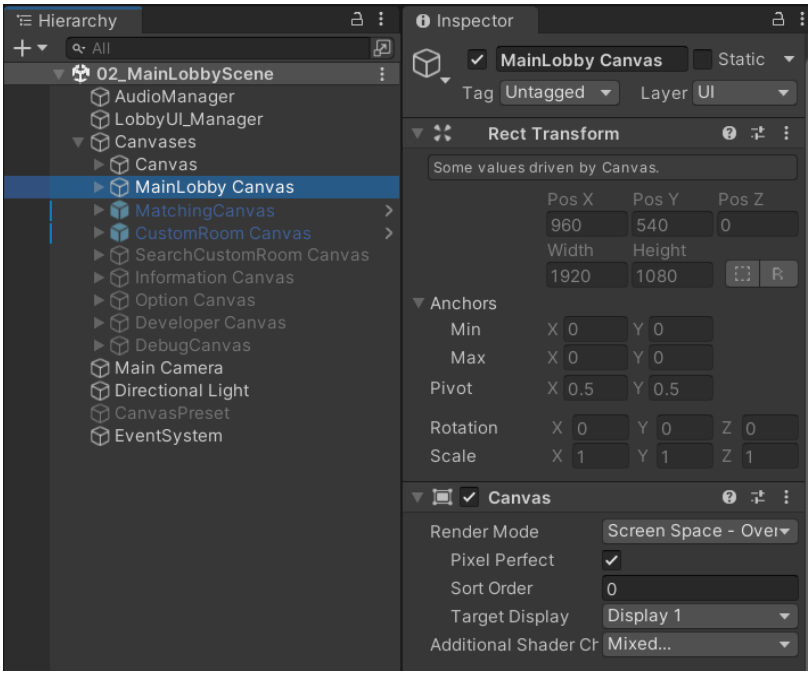
상호작용이 없는 UI일 경우



상호작용이 있는 UI일 경우

2.3. Canvas 카메라 설정

Render Camera필드를 공백으로 두면 Unity가 자동으로 채워 넣어 불필요한 리소스가 소모됨. 이를 방지하기 위해 RenderMode를 Screen Space - Overlay로 설정함.


Hierarchy
All
02_MainLobbyScene
AudioManager
LobbyUI_Manager
Canvases
Canvas
MainLobby Canvas
MatchingCanvas
CustomRoom Canvas
SearchCustomRoom Canvas
Information Canvas
Option Canvas
Developer Canvas
DebugCanvas
Main Camera
Directional Light
CanvasPreset
EventSystem
Inspector
MainLobby Canvas
Static
Tag Untagged
Layer UI
Rect Transform
Some values driven by Canvas.
Pos X
Pos Y
Pos Z
960
540
0
Width
Height
1920
1080
Anchors
Min X 0 Y 0
Max X 0 Y 0
Pivot X 0.5 Y 0.5
Rotation X 0 Y 0 Z 0
Scale X 1 Y 1 Z 1
Canvas
Render Mode
Screen Space - Over
Pixel Perfect
Sort Order
0
Target Display
Display 1
Additional Shader Ch
Mixed...

3. Object 상호작용 시스템

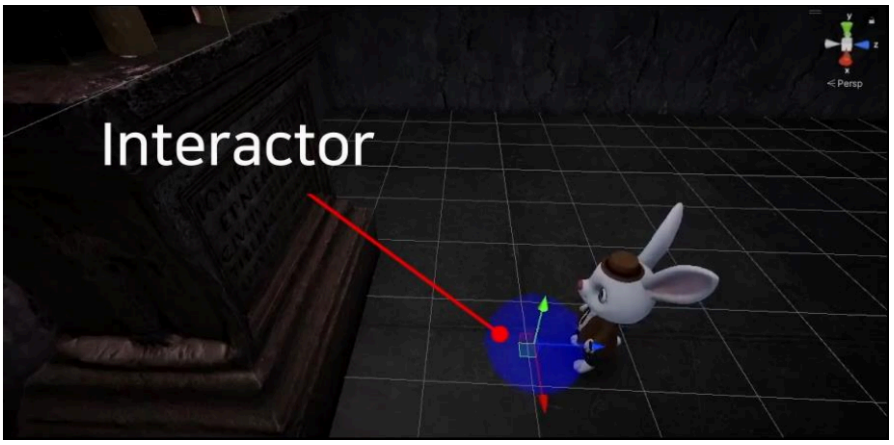
3.1. 개발 의도

- 플레이어와 다른 오브젝트가 상호작용하기 위해 개발함.

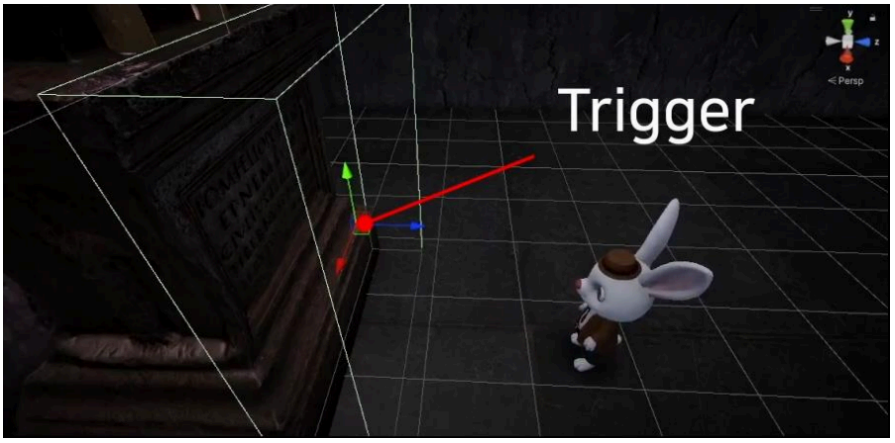
3.2. 시스템

3.2.1. 시스템 설계

- 오브젝트의 앞에는 Trigger를 위한 Collider가 존재함.
- 플레이어의 전면에는 Interactor가 존재함.
- Collider와 Interactor가 Trigger 했을 경우, 상호작용이 가능한지 확인하는 방식으로 설계함.



플레이어 앞에 존재하는 Interactor



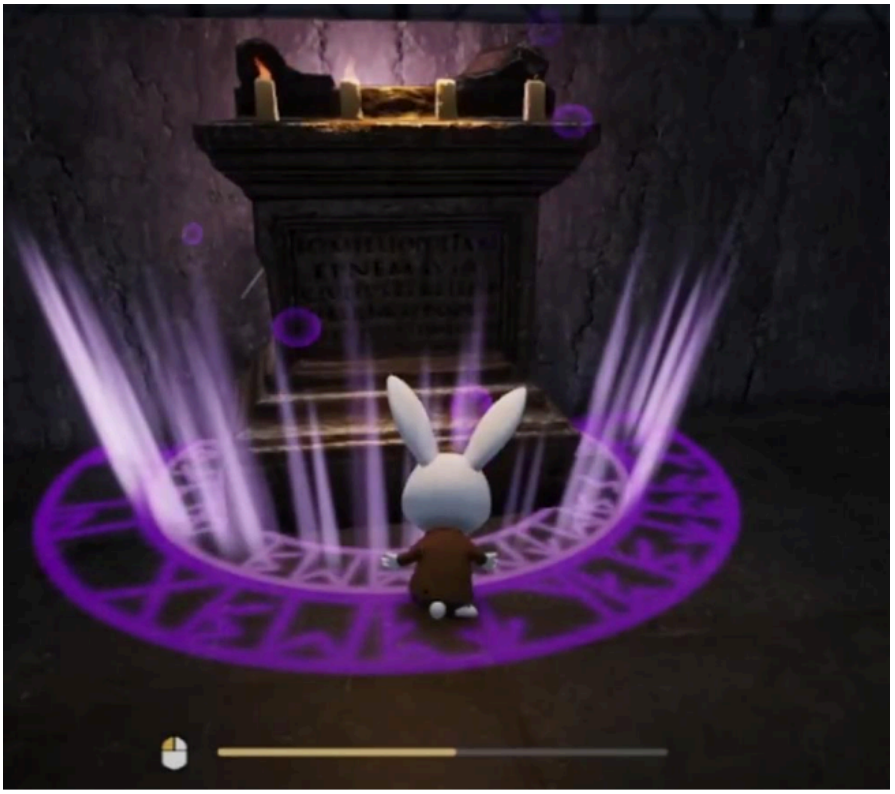
오브젝트 근처에 존재하는 Collider

- 상호작용이 가능한 경우 UI와 외곽선을 이용하여 플레이어가 상호작용 가능 여부를 확인할 수 있도록함.



상호작용이 가능한 오브젝트의 모습

- 상호작용 버튼을 눌러 상호작용 시스템을 작동하면 오브젝트 마다 정해진 이펙트와 UI를 통해 플레이어에게 상호작용을 진행하고 있다는 것을 알림.



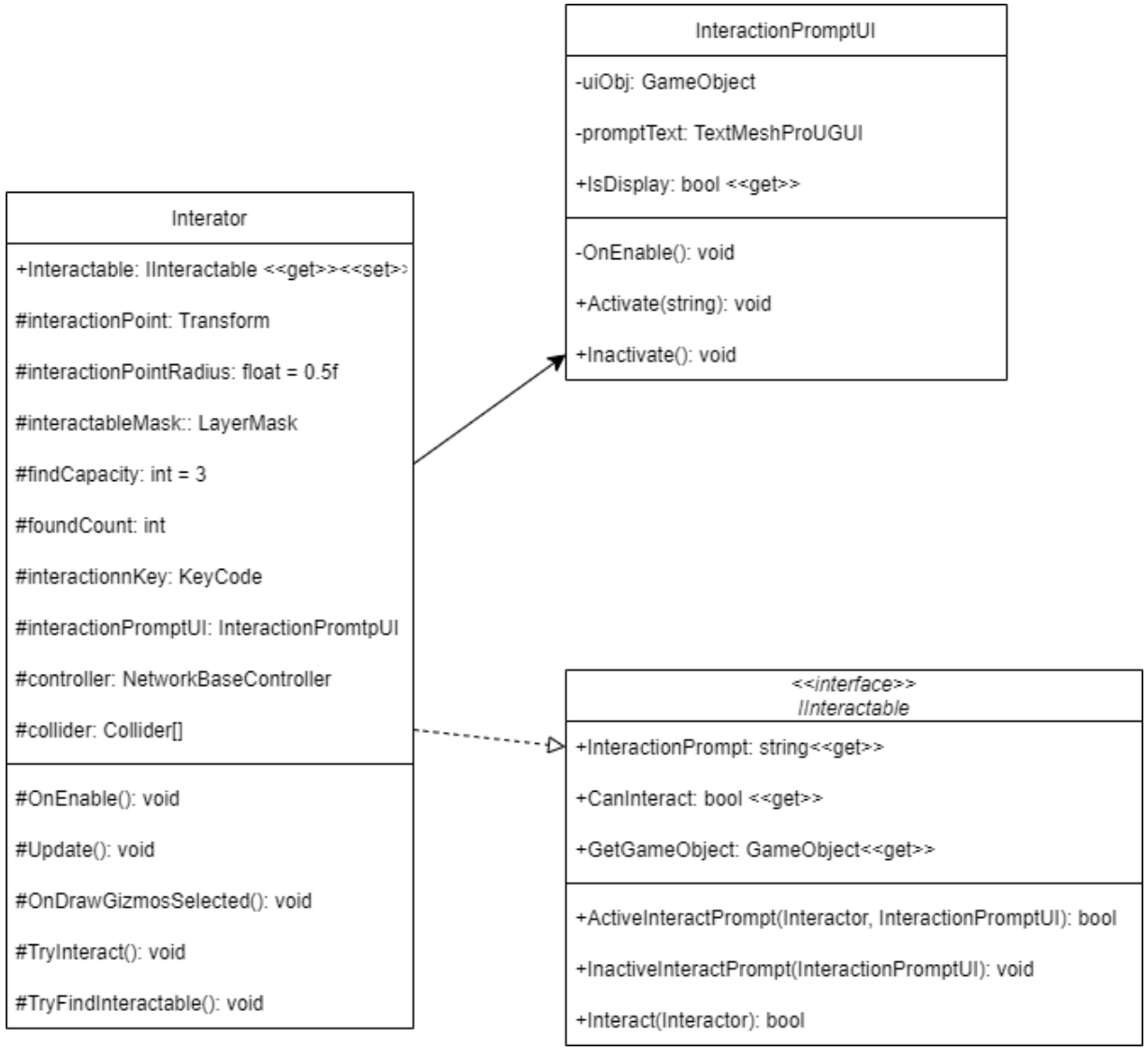
상호작용 시스템을 작동 중인 모습

- 상호작용이 끝났거나 불가능한 경우 UI와 외곽선을 표시하지 않아 플레이어에게 상호작용이 불가능함을 알림.



상호작용 시스템을 작동할 수 없을 때의 모습

3.3. 클래스 다이어그램


InteractionPromptUI
-uiObj: GameObject
-promptText: TextMeshProUGUI
+IsDisplay: bool <<get>>
-OnEnable(): void
+Activate(string): void
+Inactivate(): void
Interator
+Interactable: IInteractable <<get>><<set>>
#interactionPoint: Transform
#interactionPointRadius: float = 0.5f
#interactableMask:: LayerMask
#findCapacity: int = 3
#foundCount: int
#interactionnKey: KeyCode
#interactionPromptUI: InteractionPromtpUI
#controller: NetworkBaseController
#collider: Collider[]
#OnEnable(): void
#Update(): void
#OnDrawGizmosSelected(): void
#TryInteract(): void
#TryFindInteractable(): void
<<interface>>
IInteractable
+InteractionPrompt: string<<get>>
+CanInteract: bool <<get>>
+GetGameObject: GameObject<<get>>
+ActiveInteractPrompt(Interactor, InteractionPromptUI): bool
+InactiveInteractPrompt(InteractionPromptUI): void
+Interact(Interactor): bool

- Interactor: 상호작용이 가능한지 판단하는 클래스
- IInteractor: Interactor가 IInteractor를 찾은 경우 TryInteract()를 호출
- InteractionPromptUI: 상호작용이 가능할 경우 상호작용이 가능하다는 UI를 출력하는 클래스

4. Casting 시스템

4.1. 개발 의도

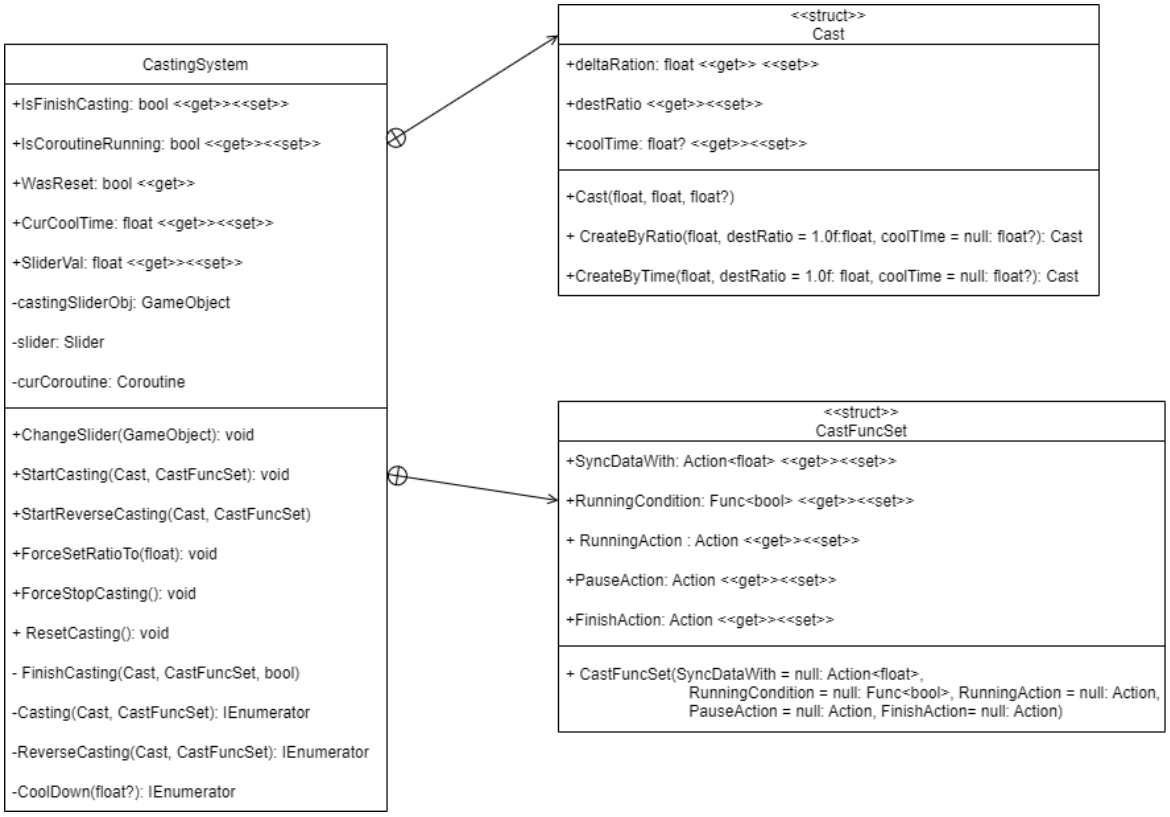
- 대부분의 상호작용이 캐스팅 후 작동하기 때문에 개발함.
- 캐스팅의 게이지를 채우기 위해서 개발함.

4.2. 시스템

4.2.1. 시스템 설계

- 캐스팅의 타입에 따라 저장 방식을 나누어 설계함.
- 캐스팅 타입
    - Auto: 한 번 누르면 끝까지 게이지 증가함.
    - Manual: 키를 누르고 있는 경우에만 게이지가 증가함.
- 캐스팅 저장 방식
    - Volatile: 캐스팅 데이터를 유지함.
    - Non-Volatile: 캐스팅 데이터를 유지하지 않음.

4.3. 클래스 다이어그램



- Cast: Cating 타입을 나타내는 클래스
- CastFuncSet: Casting을 수행할 때 이루어질 함수들의 조합인 구조체
- CastingSystem: 게이지를 충전하기 위한 시스템

Windows API를 이용한 쿠키런 모작



장르: 러닝 액션

플레이 인원 수: 1인

사용 언어 및 툴
C++

프로젝트 기간

2022년 중반기

프로젝트 인원

1인

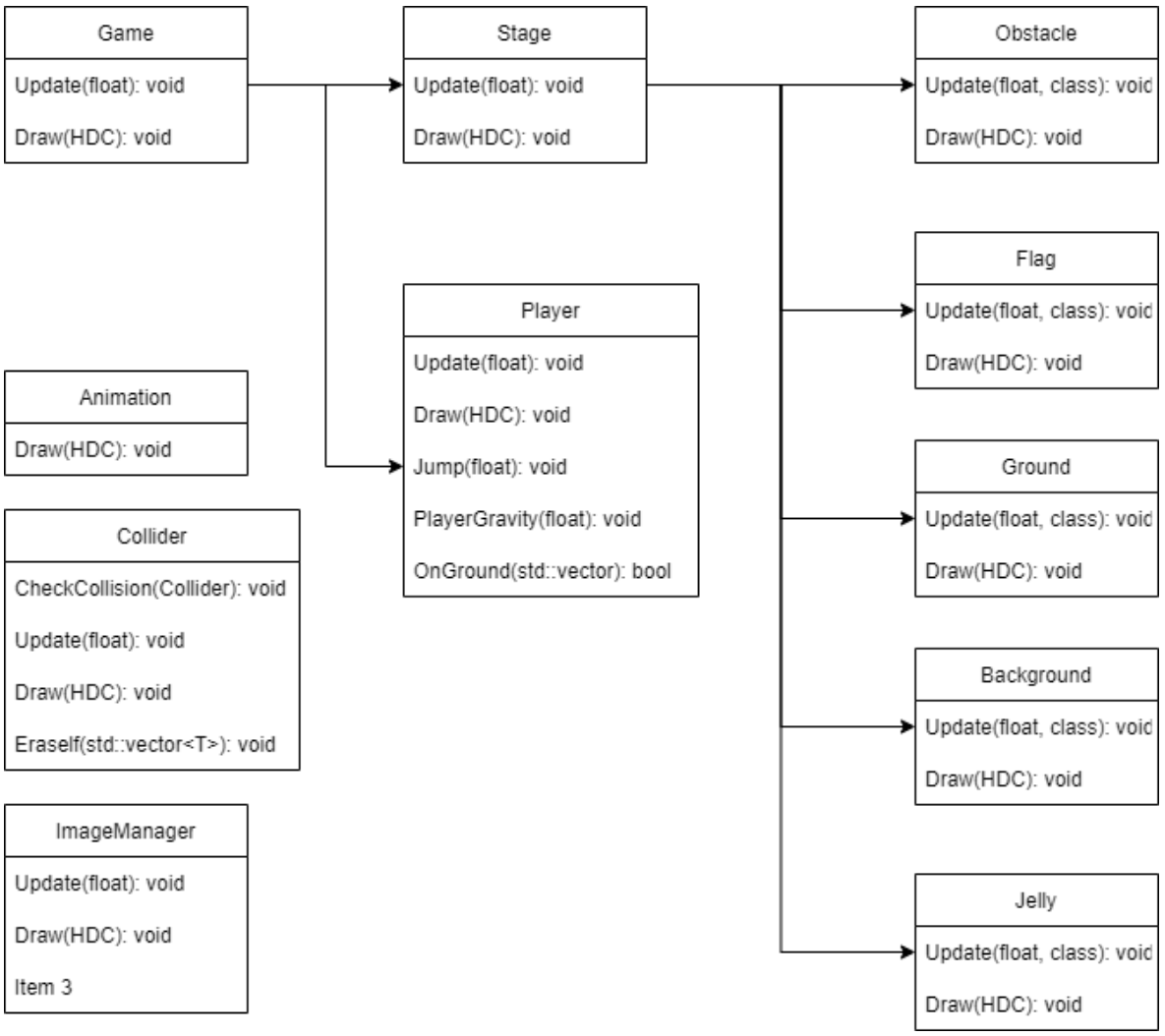
기획 의도

- 2D 횡스크롤 게임 제작
- 쿠키런 모작
- 쿠키런 1스테이지 제작 목표

주 구현 시스템

- 무한 배경을 위한 스크롤링 시스템

구성



쿠키런 게임의 구성도

- **Game**: 게임을 총괄하는 역할을 하며 Update와 Draw함수를 통해 전체적으로 게임을 관리하는 클래스
- **Stage**: Stage를 구성하는 구성요소(Obstacle, Flag, Ground, Background, Jelly)를 그리고 업데이트하는 클래스
- **Player**: 플레이어 관련 클래스

- **Background**: 게임의 배경 클래스
- **ImageManager**: 게임 이미지 클래스
- **Collider**: 충돌 처리를 위한 클래스
- **Animation**: 게임에 필요한 애니메이션을 관리하는 클래스

```
class Game
{
	enum class GameMode
	{
		GameStart,
		MainGame,
		GameEnd
	};
public:
	Game()
		:
		player({146, 278}, {59,66})
	{}
	void Update()
	{
		switch (mode)
		{
		case Game::GameMode::GameStart:
			break;
		case Game::GameMode::MainGame:
		{
				float dt = frametimer.Mark();
				if(startcheck)
				{

					player.Update(dt, *this);
					stage.Update(dt, *this);
				}
				else
				{
					InputStartKey();
				}
		}
		break;
		case Game::GameMode::GameEnd:
			break;
		default:
```

```
                    break;
                }
        }
        void Draw(HDC hdc)
        {
                switch (mode)
                {
                case Game::GameMode::GameStart:
                        break;
                case Game::GameMode::MainGame:
                        {
                                if (startcheck)
                                {
                                        HDC hMemDC;
                                        HBITMAP hOldbitmap;

                                        hMemDC = CreateCompatibleDC( hdc );
                                        if (hDoubleBufferImage == NULL)
                                        {
                                                hDoubleBufferImage =
CreateCompatibleBitmap( hdc, clientRect.right, clientRect.bottom );
                                        }
                                        hOldbitmap = (HBITMAP)SelectObject(
hMemDC, hDoubleBufferImage );

                                        FillRect( hMemDC, &clientRect,
(HBRUSH)GetStockObject( WHITE_BRUSH ) );

                                        //insert here draw to game
                                        stage.Draw( hMemDC );
                                        player.Draw( hMemDC );

                                        const std::wstring scoreStr = L"Score: "
+ std::to_wstring( score );
                                        TextOut( hMemDC, 310, 10,
scoreStr.c_str(), scoreStr.size() );

                                        BitBlt( hdc, 0, 0, clientRect.right,
clientRect.bottom, hMemDC, 0, 0, SRCCOPY );
                                        SelectObject( hMemDC, hOldbitmap );
                                        DeleteObject( hMemDC );
                                }
                                else
                                {
                                        img.Draw4( hdc ,filename);
                                        const std::wstring str = L"Press Enter
to Start!";
                                        TextOut( hdc, clientRect.right * 0.5 -
80, clientRect.bottom * 0.5 + 20, str.c_str(), str.size() );
                                }
```

```
                }
                break;
            case Game::GameMode::GameEnd:
                {
                    FillRect( hdc, &clientRect,
(HBRUSH)GetStockObject( WHITE_BRUSH ) );
                    const std::wstring str = L"Result";
                    TextOut( hdc, clientRect.right * 0.5 - 80,
clientRect.bottom * 0.5, str.c_str(), str.size() );
                    const std::wstring scoreStr = L"Score: " +
std::to_wstring( score );
                    TextOut( hdc, clientRect.right * 0.5 - 80,
clientRect.bottom * 0.5 + 20, scoreStr.c_str(), scoreStr.size() );
                }

                break;
            default:
                break;
            }
        }

        const Player& GetPlayer() const
        {
            return player;
        }
        void IsGameEnd()
    {
            mode = GameMode::GameEnd;
    }
        void playerDemage()
        {
            playerhealth -= 1;
            if (playerhealth == 0)
            {
                mode = GameMode::GameEnd;
            }
        }
        void GetScore()
        {
            score += 100;
        }
public:
        RECT clientRect;

private:
        void DrawStr(HDC hdc, const Vec2<int>& pos, const std::wstring str)
        {
            TextOut(hdc, pos.x, pos.y, str.c_str(), (int)str.size());
        }
        void InputStartKey()
```

```
	{
		if (GetAsyncKeyState( VK_RETURN ) & 0x8000)
		{
			startcheck = true;
		}

	}
private:
	const Vec2<int> corposition = { 0, 0 };
	bool startcheck = false;
	GameMode mode = GameMode::MainGame;
	FrameTimer frametimer;
	Player player;
	HBITMAP hDoubleBufferImage;

	std::wstring playerID;
	std::map<int, std::vector<std::wstring>> rank;

	Stage stage;
	int playerhealth = 1;
	int score;

	ImageManager img;
	std::wstring filename = L"resource/bg/logo.png";
};
```

Game.h의 예시

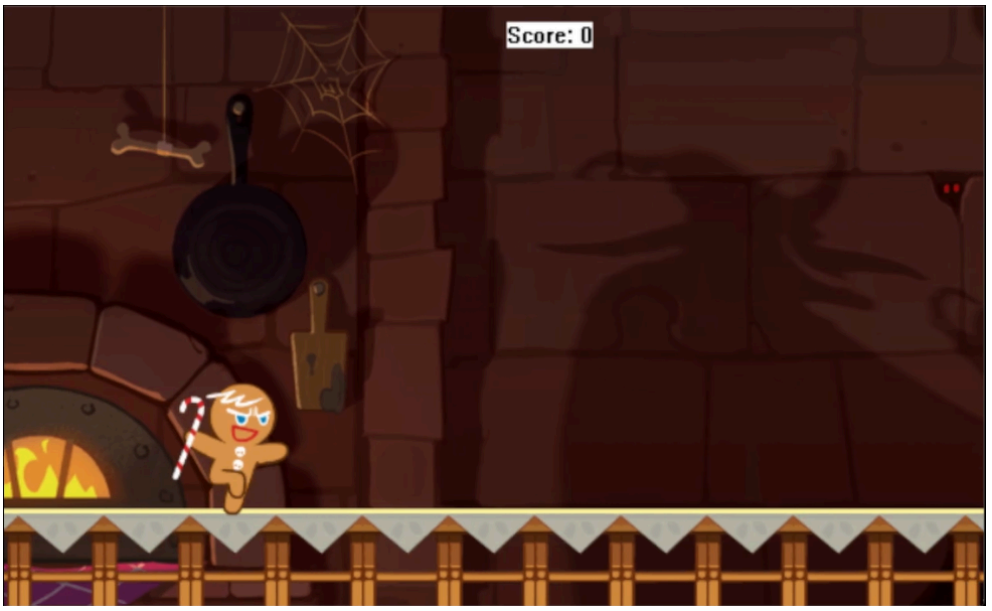
1. 무한 배경을 위한 스크롤링 시스템

1.1. 개발 의도

배경의 이미지가 끊기지 않고 적은 양의 이미지로만 배경이 무한히 이어지는 것처럼 보이기 위해 개발함.

1.2. 시스템

서로 같은 이미지 2개를 가지고 이미지의 가로 길이 만큼 이동했다면 위치를 이동시키는 방식으로 스크롤링 시스템을 개발함.



게임 내 스크롤링 적용

```
#pragma once
#include "framework.h"

const TCHAR BackgroundImagePath[] = _TEXT(
"resource/map/land0001_tm001/land0001_tm001_bg1.bmp" );

class BackGround
{
public:
```

```
	BackGround()
		:
		x(0),
		y(0),
		hBackImage((HBITMAP)LoadImage(NULL,
BackgroundImagePath,
			IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE |
LR_CREATEDIBSECTION))
	{
		GetObject(hBackImage, sizeof(BITMAP),
&bitBack);
		bx = bitBack.bmWidth;
		by = bitBack.bmHeight;
	}
	void Update(float dt)
	{
		x -= 1;
		if (x == -bx)
		{
			x = 0;
		}
	}
	void Draw(HDC hdc)
	{
		HDC hMemDC;
		HBITMAP hOldBitmap;


		CreateCompatibleDCAndSelectBitmap( hdc,
hBackImage, hMemDC, hOldBitmap );

		BitBlt(hdc, x, y, bx, by, hMemDC, 0, 0,
SRCCOPY);
		BitBlt(hdc, x + bx, y, bx, by, hMemDC, 0, 0,
SRCCOPY);

		DeleteCompatibleDC( hMemDC, hOldBitmap );
	}
private:
	int x; int y;
	HBITMAP hBackImage;
	BITMAP bitBack;
```

```
      int bx = bitBack.bmWidth;
      int by = bitBack.bmHeight;

      void CreateCompatibleDCAndSelectBitmap( HDC hdc,
HBITMAP hBitmap, HDC& hMemDC, HBITMAP& hOldBitmap )
      {
            hMemDC = CreateCompatibleDC( hdc );
            hOldBitmap = (HBITMAP)SelectObject( hMemDC,
hBitmap );
      }

      void DeleteCompatibleDC( HDC hMemDC, HBITMAP
hBitmap )
      {
            SelectObject( hMemDC, hBitmap );
            DeleteDC( hMemDC );
      }
};
```

Background.h의 코드

NIGHTMARE



장르: vr탈출게임

플레이 인원: 1인

엔진: Unity 3D

언어: C#

프로젝트 기간

2017년 상반기

프로젝트 인원

3인

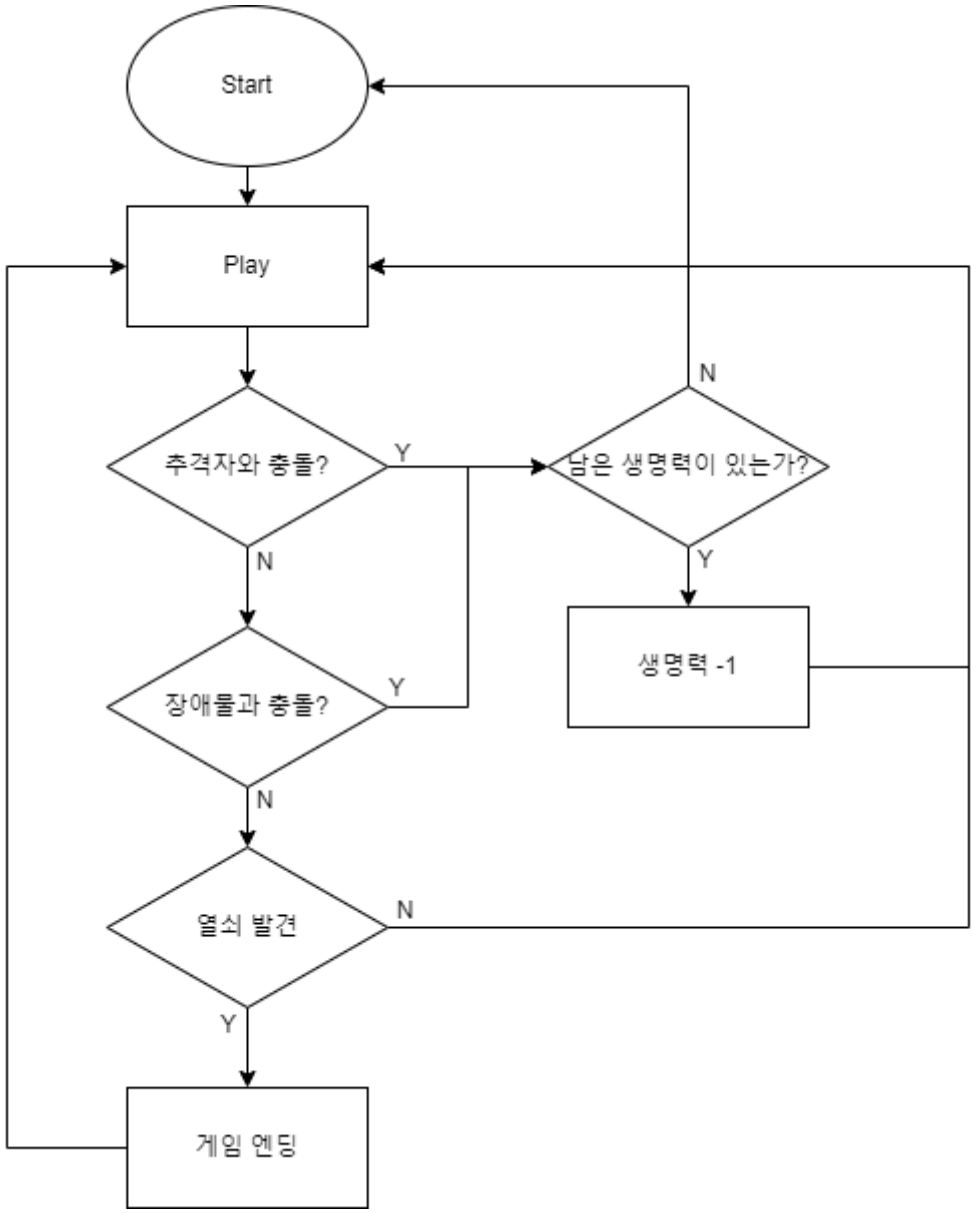
기획 의도

Unity를 이용한 VR 게임 제작
먹잇감을 찾아 돌아다니는 꿈속의 몬스터를 피해 보물 상자를 찾아서 악몽에서 탈출하는 게임

주 구현 시스템

- Raycast를 이용한 몬스터의 트래킹 시스템

구성



- 출발한 플레이어가 추격자와 충돌했는지 여부를 판단함.
- 충돌했다면 남은 생명 오브젝트 수에 따라 GameOver 화면으로 이동할지, 생명 오브젝트만 -1 할지 결정함.
- 장애물 충돌 시에도 이와 동일하게 남은 생명 오브젝트 개수에 따라 알고리즘을 구성함.
- 플레이어가 게임을 탈출할 수 있는 열쇠를 발견했는지 판단하여,
- 발견했다면 Game Clear 화면으로, 발견하지 않았다면 계속 Play를 하도록 알고리즘을 구성함.

1. RayCast를 이용한 몬스터 트래킹 시스템

1.1. 개발의도

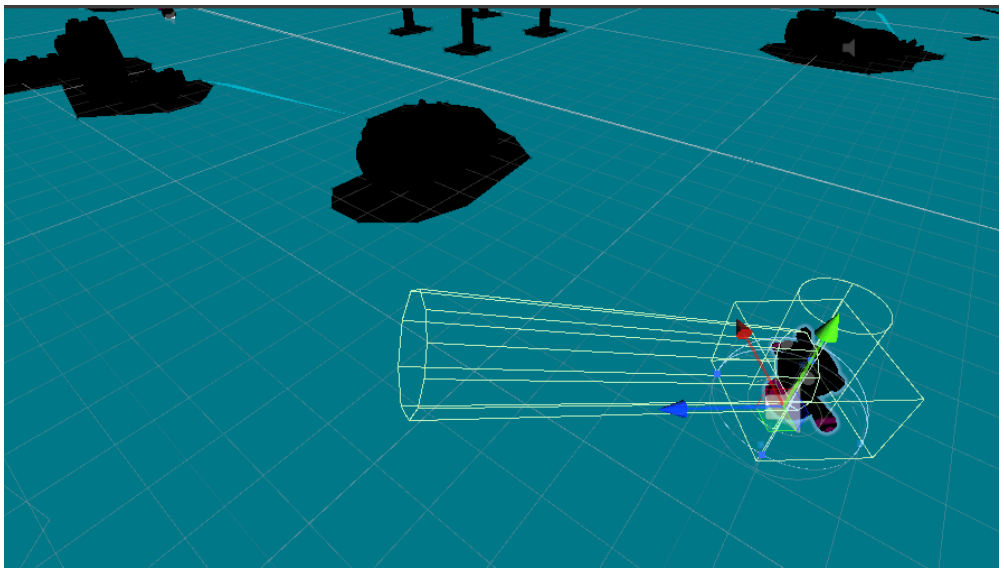
- 플레이어에게 좀 더 게임에 대한 긴박감을 느낄 수 있도록 기획.
- 한번 쫓기게된다면 플레이어가 게임오버 상태가 될 때 까지 쫓아오도록 개발함.



게임 속 몬스터

1.2. 시스템

몬스터는 NavMesh와 Raycast를 사용함.



몬스터는 상태를 가지고 있으며 아래와 같은 상태의 종류를 가지고 있음.

몬스터의 state : idle, walk, search,chase

idle: 몬스터의 대기 상태

walk: 몬스터가 걷는 상태

search: 몬스터가 플레이어를 찾는 상태

chase: 플레이어를 발견한 몬스터가 플레이어를 쫓는 상태

몬스터의 눈 위치에서 ray를 보내어 충돌하는 오브젝트를 확인함.
ray와 충돌한 오브젝트가 “Player”라면 인형의 state를 chase로 변경하고 플레이어를 추적함.